



标签过滤 UI 测试说明

本文档面向通过 UI 测试标签过滤功能的测试人员，说明两类 LLRP 过滤动作参数如何从 UI 配置下发到读写器，以及对实际寻卡结果的影响。

一、整体结构

标签过滤最终下发在 `ROSpec` 的盘点命令里，结构如下：

```
ADD_ROSPEC
  ROSpec
    AISpec
      InventoryParameterSpec
        AntennaConfiguration
          C1G2InventoryCommand
            TagInventoryStateAware
              C1G2Filter[]
                C1G2TagInventoryMask
                C1G2TagInventoryStateUnawareFilterAction
                或 C1G2TagInventoryStateAwareFilterAction
            C1G2SingulationControl
              Session
              C1G2TagInventoryStateAwareSingulationAction
```

每个 `C1G2Filter` 都包含两部分：

部分	作用
<code>C1G2TagInventoryMask</code>	定义匹配什么标签，例如 EPC/TID/User 的哪个 bit 段等于哪个 mask
<code>FilterAction</code>	定义匹配/不匹配后如何改变标签状态

当前 UI 将过滤动作分成两种模式：

UI Filter Mode	LLRP 参数
OnlyFilter1 / OnlyFilter2 / Filter1AndFilter2 / Filter1OrFilter2 / UseTagSelectFilters	C1G2TagInventoryStateUnawareFilterAction
UseStateAwareTagSelectFilters	C1G2TagInventoryStateAwareFilterAction

二、基础概念和枚举含义

1. MemoryBank

MemoryBank 表示 filter 到哪个标签内存区里匹配 TagMask。

UI/SDK 枚举	含义
Epc	EPC 区，常用于按 EPC 前缀或完整 EPC 过滤
Tid	TID 区，常用于按芯片型号、厂商信息过滤
User	User 区，常用于按业务写入数据过滤
Reserved	Reserved 区，通常不建议用于盘点过滤

2. StateUnawareAction

StateUnawareAction 是普通 C1G2TagInventoryStateUnawareFilterAction 使用的动作。它不指定 Target，可以理解为对标签的普通 Select 状态做处理。

枚举	含义
Select	将标签设为选中状态
Unselect	将标签设为未选中状态
DoNothing	不改变标签当前状态

在 UseTagSelectFilters 模式里，UI 会让测试人员分别选择：

MatchAction
NonMatchAction

SDK 会把二者组合成 LLRP 的 `ENUM_C1G2StateUnawareAction` :

LLRP 枚举	匹配标签	不匹配标签
<code>Select_Unselect</code>	Select	Unselect
<code>Select_DoNothing</code>	Select	不改变
<code>DoNothing_Unselect</code>	不改变	Unselect
<code>Unselect_DoNothing</code>	Unselect	不改变
<code>Unselect_Select</code>	Unselect	Select
<code>DoNothing_Select</code>	不改变	Select

3. SL

SL 是 Gen2 标签里的 Selected Flag, 可以粗略理解为一个“是否被选中”的标志位。

常见用法:

匹配过滤条件的标签 -> SL
不匹配过滤条件的标签 -> Not_SL
读写器只寻 SL 标签

这样最终就只读到匹配过滤条件的标签。

4. Session 和 A/B

Gen2 标签有 4 个 inventory session:

S0
S1
S2
S3

每个 Session 里都有一个 inventoried state:

A
B

可以把它理解成每个标签在每个 Session 下都有一个 A/B 状态。读写器盘点时可以指定:

```
Session = S1  
InventoryTarget = A
```

含义是:

本轮按 S1 盘点, 只让 S1/A 的标签响应

如果设置为:

```
Session = S1  
InventoryTarget = B
```

则本轮只让 S1/B 的标签响应。

5. InventoryTarget

InventoryTarget 对应 LLRP 的 C1G2TagInventoryStateAwareSingulationAction.I。

UI/SDK 枚举	LLRP 值	含义
A	State_A	寻当前 Session 下 A 状态的标签
B	State_B	寻当前 Session 下 B 状态的标签

注意：InventoryTarget 必须结合 Session 理解。A 不是全局 A，而是当前 Session 下的 A。

6. InventorySearchMode

InventorySearchMode 对应 LLRP 的 C1G2TagInventoryStateAwareSingulationAction.S。

UI/SDK 枚举	LLRP 值	含义
SL	SL	寻 SL 标签
Not_SL	Not_SL	寻非 SL 标签

当状态感知 filter 使用 StateAwareTarget = SL 时，InventorySearchMode 是关键筛选条件。

例如：

```
StateAwareTarget = SL
StateAwareAction = AssertSLOrA_DeassertSLOrB
InventorySearchMode = SL
```

含义：

```
匹配 mask 的标签 -> SL
不匹配 mask 的标签 -> Not_SL
读写器寻 SL
最终读到匹配 mask 的标签
```

7. StateAwareTarget

StateAwareTarget 对应 LLRP 的 C1G2TagInventoryStateAwareFilterAction.Target，表示 filter action 要改哪个状态。

LLRP 枚举	含义
SL	修改标签的 SL / Not_SL 状态
Inventoried_State_For_Session_S0	修改 S0 的 A/B 状态
Inventoried_State_For_Session_S1	修改 S1 的 A/B 状态

LLRP 枚举	含义
Inventoried_State_For_Session_S2	修改 S2 的 A/B 状态
Inventoried_State_For_Session_S3	修改 S3 的 A/B 状态

如果 Target = SL，action 名字里的 SL 生效。

如果 Target = S0/S1/S2/S3，action 名字里的 A/B 生效。

8. StateAwareAction

StateAwareAction 对应 LLRP 的 C1G2TagInventoryStateAwareFilterAction.Action，表示匹配/不匹配时如何修改 StateAwareTarget 指定的状态。

名称格式是：

匹配时动作_不匹配时动作

常用枚举含义：

LLRP 枚举	匹配标签	不匹配标签
AssertSL0rA_DeassertSL0rB	设为 SL 或 A	设为 Not_SL 或 B
AssertSL0rA_Noop	设为 SL 或 A	不改变
Noop_DeassertSL0rB	不改变	设为 Not_SL 或 B
NegateSL0rABBA_Noop	SL 取反，或 A/B 互换	不改变
DeassertSL0rB_AssertSL0rA	设为 Not_SL 或 B	设为 SL 或 A
DeassertSL0rB_Noop	设为 Not_SL 或 B	不改变
Noop_AssertSL0rA	不改变	设为 SL 或 A
Noop_NegateSL0rABBA	不改变	SL 取反，或 A/B 互换

举例：

```
StateAwareTarget = SL
StateAwareAction = AssertSLOrA_DeassertSLOrB
```

实际含义：

```
匹配标签 -> Assert SL
不匹配标签 -> Deassert SL
```

如果换成：

```
StateAwareTarget = Inventoried_State_For_Session_S1
StateAwareAction = AssertSLOrA_DeassertSLOrB
```

实际含义：

```
匹配标签 -> S1/A
不匹配标签 -> S1/B
```

9. 测试时怎么记

普通过滤只记：

```
Mask 决定匹配谁
Action 决定匹配/不匹配后 Select 还是 Unselect
```

状态感知过滤要记：

```
Mask 决定匹配谁
StateAwareTarget 决定改 SL 还是某个 Session 的 A/B
StateAwareAction 决定匹配/不匹配后怎么改
Session + InventoryTarget + InventorySearchMode 决定 reader 最终寻哪些状态的标签
```

三、普通过滤：

C1G2TagInventoryStateUnawareFilterAction

1. UI 设置入口

页面： `盘点配置`

测试普通过滤时， `Filter Mode` 选择以下任一种：

```
OnlyFilter1
OnlyFilter2
Filter1AndFilter2
Filter1OrFilter2
UseTagSelectFilters
```

2. UI 参数和 LLRP 参数对应关系

UI 参数	LLRP 参数	说明
<code>MemoryBank</code>	<code>C1G2TagInventoryMask.MB</code>	匹配 EPC、TID、User 等内存区
<code>BitPointer</code>	<code>C1G2TagInventoryMask.Pointer</code>	从指定内存区哪个 bit 开始匹配
<code>BitCount</code>	<code>C1G2TagInventoryMask.TagMask</code> 截断长度	大于 0 时，只使用 mask 前 <code>BitCount</code> 个 bit
<code>TagMask</code>	<code>C1G2TagInventoryMask.TagMask</code>	十六进制匹配值
<code>FilterOp</code>	<code>C1G2TagInventoryStateUnawareFilterAction.Action</code>	用于 <code>OnlyFilter1/2/AND/OR</code>
<code>MatchAction</code>	<code>C1G2TagInventoryStateUnawareFilterAction.Action</code> 的前半段	用于 <code>UseTagSelectFilters</code>

UI 参数	LLRP 参数	说明
NonMatchAction	C1G2TagInventoryStateUnawareFilterAction.Action 的后半段	用于 UseTagSelectFilters

C1G2TagInventoryStateUnawareFilterAction 只有 Action，没有 Target。

常见 Action 含义：

Action	匹配标签	不匹配标签
Select_Unselect	Select	Unselect
Select_DoNothing	Select	不改变
DoNothing_Unselect	不改变	Unselect
Unselect_DoNothing	Unselect	不改变
Unselect_Select	Unselect	Select
DoNothing_Select	不改变	Select

3. 各 UI 模式的下发和寻卡预期

OnlyFilter1

UI 示例：

```
Filter Mode = OnlyFilter1
Filter1.MemoryBank = EPC
Filter1.BitPointer = 32
Filter1.TagMask = E200
Filter1.FilterOp = Match
```

应下发：

```
C1G2Filter[0]
  C1G2TagInventoryMask
    MB = EPC
    Pointer = 32
    TagMask = E200
  C1G2TagInventoryStateUnawareFilterAction
    Action = Select_Unselect
```

读写器寻卡行为：

EPC 指定位置匹配 E200 的标签 -> Select
不匹配 E200 的标签 -> Unselect
后续寻卡只返回被 Select 的标签

预期结果：只读到匹配 Filter1 的标签。

OnlyFilter2

UI 示例：

```
Filter Mode = OnlyFilter2
Filter2.MemoryBank = TID
Filter2.BitPointer = 0
Filter2.TagMask = E280
Filter2.FilterOp = Match
```

应下发：

```
C1G2Filter[0]
  C1G2TagInventoryMask
    MB = TID
    Pointer = 0
    TagMask = E280
  C1G2TagInventoryStateUnawareFilterAction
    Action = Select_Unselect
```

读写器寻卡行为：

```
TID 匹配 E280 的标签 -> Select
TID 不匹配 E280 的标签 -> Unselect
```

预期结果：只读到匹配 Filter2 的标签。

如果 `FilterOp = NotMatch`，应下发 `Action = Unselect_Select`，预期结果变为只读到不匹配 Filter2 的标签。

Filter1AndFilter2

UI 示例：

```
Filter Mode = Filter1AndFilter2
Filter1 = EPC / Pointer 32 / Mask E200 / Match
Filter2 = TID / Pointer 0 / Mask E280 / Match
```

应下发：

```
C1G2Filter[0]
  Mask = Filter1
  C1G2TagInventoryStateUnawareFilterAction.Action = Select_Unselect

C1G2Filter[1]
  Mask = Filter2
  C1G2TagInventoryStateUnawareFilterAction.Action = DoNothing_Unselect
```

读写器寻卡行为：

```
第一步 Filter1:
  命中 Filter1 -> Select
  未命中 Filter1 -> Unselect

第二步 Filter2:
  命中 Filter2 -> 不改变已有状态
  未命中 Filter2 -> Unselect
```

预期结果：只有同时命中 Filter1 和 Filter2 的标签保持 Select，最终被读到。

Filter1OrFilter2

UI 示例:

```
Filter Mode = Filter1OrFilter2
Filter1 = EPC / Pointer 32 / Mask E200 / Match
Filter2 = TID / Pointer 0 / Mask E280 / Match
```

应下发:

```
C1G2Filter[0]
  Mask = Filter1
  C1G2TagInventoryStateUnawareFilterAction.Action = Select_Unselect

C1G2Filter[1]
  Mask = Filter2
  C1G2TagInventoryStateUnawareFilterAction.Action = Select_DoNothing
```

读写器寻卡行为:

```
第一步 Filter1:
  命中 Filter1 -> Select
  未命中 Filter1 -> Unselect

第二步 Filter2:
  命中 Filter2 -> Select
  未命中 Filter2 -> 不改变已有状态
```

预期结果: 命中 Filter1 或命中 Filter2 的标签会被读到。

UseTagSelectFilters

UI 示例:

```
Filter Mode = UseTagSelectFilters
```

新增过滤条件：

```
MemoryBank = EPC
```

```
BitPointer = 32
```

```
TagMask = E200
```

```
MatchAction = Select
```

```
NonMatchAction = Unselect
```

应下发：

```
C1G2Filter[0]
```

```
C1G2TagInventoryMask
```

```
MB = EPC
```

```
Pointer = 32
```

```
TagMask = E200
```

```
C1G2TagInventoryStateUnawareFilterAction
```

```
Action = Select_Unselect
```

读写器寻卡行为：

匹配该过滤条件的标签 -> Select

不匹配该过滤条件的标签 -> Unselect

预期结果：按每条过滤条件的 `MatchAction` / `NonMatchAction` 改变标签 Select 状态，再进行寻卡。

测试注意：

- `UseTagSelectFilters` 只测试 `C1G2TagInventoryStateUnawareFilterAction`。
- 此模式下 UI 不应显示 `StateAwareTarget` / `StateAwareAction`。

四、状态感知过滤： C1G2TagInventoryStateAwareFilterAction

1. UI 设置入口

状态感知过滤涉及两个页面。

页面一： 参数配置

需要先确认并设置：

设备能力 CanDoTagInventoryStateAwareSingulation = True
InventoryStateAware = 开启
Session = S0/S1/S2/S3
InventoryTarget = A 或 B
InventorySearchMode = SL 或 Not_SL

页面二： 盘点配置

Filter Mode = UseStateAwareTagSelectFilters
新增过滤条件：
MemoryBank
BitPointer
BitCount
TagMask
StateAwareTarget
StateAwareAction

如果 InventoryStateAware 未开启，或者读写器能力不支持，保存/下发时应失败，不应静默降级为普通过滤。

2. UI 参数和 LLRP 参数对应关系

UI 参数	LLRP 参数	说明
InventoryStateAware	C1G2InventoryCommand.TagInventoryStateAware	是否启用状态感知盘点

UI 参数	LLRP 参数	说明
Session	C1G2SingulationControl.Session	使用哪个 Gen2 Session
InventoryTarget	C1G2TagInventoryStateAwareSingulationAction.I	寻 State_A 或 State_B
InventorySearchMode	C1G2TagInventoryStateAwareSingulationAction.S	寻 SL 或 Not_SL
MemoryBank	C1G2TagInventoryMask.MB	匹配内存区
BitPointer	C1G2TagInventoryMask.Pointer	匹配起始 bit
BitCount	C1G2TagInventoryMask.TagMask 截断长度	大于 0 时截断 mask
TagMask	C1G2TagInventoryMask.TagMask	匹配值
StateAwareTarget	C1G2TagInventoryStateAwareFilterAction.Target	要操作 SL, 还是 S0/S1/S2/S3 的 A/B 状态
StateAwareAction	C1G2TagInventoryStateAwareFilterAction.Action	匹配/ 不匹配时如何改变目标状

3. 应下发的 LLRP 结构

示例 UI:

参数配置:

InventoryStateAware = 开启

Session = S1

InventoryTarget = A

InventorySearchMode = SL

盘点配置:

Filter Mode = UseStateAwareTagSelectFilters

MemoryBank = EPC

BitPointer = 32

TagMask = E200

StateAwareTarget = SL

StateAwareAction = AssertSLOrA_DeassertSLOrB

应下发:

```
C1G2InventoryCommand
  TagInventoryStateAware = True
  C1G2Filter[0]
    C1G2TagInventoryMask
      MB = EPC
      Pointer = 32
      TagMask = E200
    C1G2TagInventoryStateAwareFilterAction
      Target = SL
      Action = AssertSLOrA_DeassertSLOrB
  C1G2SingulationControl
    Session = S1
    C1G2TagInventoryStateAwareSingulationAction
      I = State_A
      S = SL
```

4. 读写器寻卡行为

状态感知过滤分两步理解。

第一步：Filter 根据 mask 改变标签状态。

如果：

```
StateAwareTarget = SL
StateAwareAction = AssertSLOrA_DeassertSLOrB
```

则：

```
匹配 mask 的标签 -> Assert SL
不匹配 mask 的标签 -> Deassert SL
```

第二步：Singulation 决定本轮寻哪些状态的标签。

如果：

```
InventorySearchMode = SL
```

则读写器本轮寻 `SL` 标签。

综合结果：

匹配 `mask` 的标签被置为 `SL`
读写器寻 `SL`
最终只读到匹配 `mask` 的标签

5. 使用 Session A/B 的示例

UI 示例：

参数配置：

```
InventoryStateAware = 开启  
Session = S1  
InventoryTarget = A  
InventorySearchMode = Not_SL
```

盘点配置：

```
Filter Mode = UseStateAwareTagSelectFilters  
MemoryBank = EPC  
BitPointer = 32  
TagMask = E200  
StateAwareTarget = Inventoried_State_For_Session_S1  
StateAwareAction = AssertSL0rA_DeassertSL0rB
```

应下发：

```
C1G2TagInventoryStateAwareFilterAction  
  Target = Inventoried_State_For_Session_S1  
  Action = AssertSL0rA_DeassertSL0rB  
  
C1G2SingulationControl  
  Session = S1  
  C1G2TagInventoryStateAwareSingulationAction  
    I = State_A  
    S = Not_SL
```

读写器寻卡行为：

1. 读写器先执行 `Select/Filter` 阶段
 - 对每个在场标签读取 `EPC` 指定位置的数据
 - 如果该位置匹配 `E200`，则执行 `StateAwareAction` 的前半段 `AssertSL0rA`
 - 因为 `Target` 是 `Inventoried_State_For_Session_S1`，所以前半段的 `A` 表示写入 `S1/A`
 - 如果该位置不匹配 `E200`，则执行 `StateAwareAction` 的后半段 `DeassertSL0rB`
 - 因为 `Target` 是 `Inventoried_State_For_Session_S1`，所以后半段的 `B` 表示写入 `S1/B`
2. 标签状态被分组
 - 匹配 `mask` 的标签 -> `S1/A`
 - 不匹配 `mask` 的标签 -> `S1/B`
3. 读写器再执行 `Inventory/Singulation` 阶段
 - `Session = S1`，表示本轮按 `S1` 的 `inventoried state` 寻卡
 - `InventoryTarget = A`，对应 LLRP 的 `I = State_A`
 - 因此只有当前处于 `S1/A` 的标签会参与响应
 - `S1/B` 的标签会被压制，不参与本轮响应

预期结果：只读到匹配 `mask` 且被置为 `S1/A` 的标签。

这里的 `InventorySearchMode = Not_SL` 会下发为 `S = Not_SL`，但本示例的核心筛选目标是 `StateAwareTarget = Inventoried_State_For_Session_S1` 与 `InventoryTarget = A`。测试时重点观察 `Session = S1` 和 `I = State_A` 是否让 `S1/A` 标签响应。

如果把 `InventoryTarget` 改成 `B`，则寻卡目标变为 `S1/B`，预期结果会反过来：不匹配 `TagMask` 的标签被读到，匹配 `TagMask` 的标签不响应。

6. 测试注意

- `UseStateAwareTagSelectFilters` 只测试 `C1G2TagInventoryStateAwareFilterAction`。
- 此模式下 UI 应显示 `StateAwareTarget / StateAwareAction`，不应显示普通 `MatchAction / NonMatchAction`。
- 此模式依赖外层 `InventoryStateAware`、`Session`、`InventoryTarget`、`InventorySearchMode`。
- 如果 filter action 把标签置为某状态，但 singulation 搜索的是另一个状态，可能出现标签在场但读不到的现象。
- 测试时建议准备至少两类标签：一类匹配 `TagMask`，一类不匹配 `TagMask`，方便确认返回

结果是否符合预期。

五、测试观察点

测试人员可以从三处确认结果：

- 1. UI 配置是否按模式显示正确字段。
- 2. LLRP 消息树中是否出现正确参数：
 - 普通过滤： C1G2TagInventoryStateUnawareFilterAction
 - 状态感知过滤： C1G2TagInventoryStateAwareFilterAction
- 3. 实际寻卡结果是否只返回预期标签。

建议重点覆盖：

测试场景	预期
OnlyFilter1 + Match	只读 Filter1 命中的标签
OnlyFilter2 + NotMatch	只读 Filter2 未命中的标签
Filter1AndFilter2	只读同时命中两个 filter 的标签
Filter1OrFilter2	只读命中任一 filter 的标签
UseTagSelectFilters	LLRP 使用 C1G2TagInventoryStateUnawareFilterAction
UseStateAwareTagSelectFilters 且状态感知开启	LLRP 使用 C1G2TagInventoryStateAwareFilterAction
UseStateAwareTagSelectFilters 但状态感知未开启	保存/下发失败，不应静默变成普通过滤